Máquinas de Soporte Vectorial (SVM)

Técnicas de Machine Learning Supervisado

Michael Lata

15 de junio de 2026

Resumen. Este informe se centra en las **Máquinas de Soporte Vectorial (SVM)**: su definición e intuición, la formulación matemática del margen máximo, la regularización mediante el parámetro C, el **truco del kernel** para fronteras no lineales, su uso en **clasificación** y **regresión (SVR)**, y sus aplicaciones. El código que aparece es **únicamente de referencia**: muestra cómo se invocaría cada método, pero no es el objetivo de esta sesión.

Contents

1 Contexto: clasificación vs. regresión **1**

2 Definición e intuición **1**

3 Formulación matemática **2**
3.1 Margen duro (datos linealmente separables) 2
3.2 Margen blando y regularización (parámetro C) 2
3.3 Forma dual y vectores de soporte (referencial) 3
3.4 Proceso interno: cómo nacen y operan los vectores de soporte 3

4 El truco del kernel (*kernel trick*) **4**

5 SVM para clasificación **4**

6 SVM para regresión (SVR) **5**

7 Aplicaciones y rol histórico **5**

8 Ventajas y desventajas **5**

9 Código de referencia **5**

Referencias **6**

1 Contexto: clasificación vs. regresión

La SVM es una técnica de **aprendizaje supervisado**: aprende a partir de datos etiquetados $\{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$. Sirve tanto para:

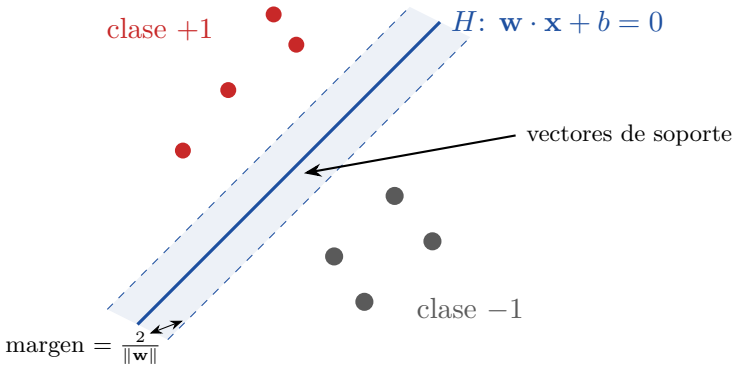
- **Clasificación** — la salida es una categoría discreta (ej. spam / no spam).
- **Regresión** — la salida es un valor numérico continuo (variante **SVR**).

2 Definición e intuición

Una **SVM** es un clasificador supervisado que, en lugar de basar la asignación de clase en distancias a otros puntos, calcula la **frontera (el hiperplano)** que mejor separa los grupos. De entre todos

los hiperplanos que separan los datos, la SVM elige aquel que **maximiza el margen**: la distancia a los puntos más cercanos de cada clase.

- Los **vectores de soporte** son los puntos que definen el margen máximo de separación. Son los **únicos** que determinan el hiperplano; los demás no influyen.
- Se llaman *vectores* (y no puntos) porque cada muestra tiene tantos elementos como dimensiones tenga el espacio de entrada: es un vector de n dimensiones.
- El término *máquina* proviene de *machine learning*.



El hiperplano H maximiza la distancia a los puntos más cercanos de ambas clases. Una muestra nueva se clasifica según el lado de H en el que caiga.

Idea clave: los problemas reales tienen muchísimas dimensiones; por eso no se busca una "línea" sino un *hiperplano* de $n-1$ dimensiones que maximice el margen.

3 Formulación matemática

El hiperplano se define como $\mathbf{w}\cdot\mathbf{x}+b=0$, y la regla de decisión es

$$f(\mathbf{x}) = \operatorname{sign}(\mathbf{w}\cdot\mathbf{x}+b).$$

La anchura del margen es $2/\|\mathbf{w}\|$, de modo que **maximizar el margen** equivale a **minimizar** $\|\mathbf{w}\|$.

3.1 Margen duro (datos linealmente separables)

$$\min_{\mathbf{w},b} \ \tfrac{1}{2}\|\mathbf{w}\|^2 \qquad \text{sujeto a} \qquad y_i(\mathbf{w}\cdot\mathbf{x}_i+b)\geq 1, \quad \forall i. \tag{1}$$

3.2 Margen blando y regularización (parámetro C)

Es frecuente que los datos tengan ruido, no estén etiquetados perfectamente o sean difíciles de separar. Para esos casos se permite que algunos puntos violen el margen, introduciendo **variables de holgura** $\xi_i \geq 0$:

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \ \tfrac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{N}\xi_i \qquad \text{s.a.} \qquad y_i(\mathbf{w}\cdot\mathbf{x}_i+b)\geq 1-\xi_i, \quad \xi_i\geq 0. \tag{2}$$

El hiperparámetro C es la **penalización del error de entrenamiento** y controla la regularización:

C **alto**	C **bajo**
Penaliza mucho los errores; el modelo se ajusta más al entrenamiento.	Margen más amplio y tolerante a errores.
Margen más estrecho.	Mejor generalización en general.
Riesgo de *overfitting*.	Riesgo de *underfitting*.

En la variante de regresión (SVR) aparece además el parámetro ε (épsilon), un **umbral/threshold**: no se penaliza la predicción mientras el error caiga por debajo de él.

3.3 Forma dual y vectores de soporte (referencial)

Resolviendo con multiplicadores de Lagrange α_i se llega al problema **dual**, que es donde aparece de forma natural el kernel:

$$\max_{\boldsymbol{\alpha}} \sum_i \alpha_i - \tfrac{1}{2} \sum_{i,j} \alpha_i \alpha_j \, y_i y_j \, K(\mathbf{x}_i, \mathbf{x}_j) \quad \text{s.a.} \quad 0 \le \alpha_i \le C, \quad \sum_i \alpha_i y_i = 0. \tag{3}$$

Solo las muestras con $\alpha_i > 0$ son **vectores de soporte**; el resto no participa en la predicción. Esto hace la solución compacta y determinista (óptimo global único).

3.4 Proceso interno: cómo nacen y operan los vectores de soporte

La pregunta clave es *¿por qué* unos puntos tienen $\alpha_i > 0$ y otros $\alpha_i = 0$? La respuesta está en las **condiciones de Karush–Kuhn–Tucker (KKT)**, que todo óptimo del problema (2)–(3) debe cumplir. La más importante es la **holgura complementaria**:

$$\alpha_i \left[\, y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1 + \xi_i \, \right] = 0, \qquad (C - \alpha_i)\, \xi_i = 0. \tag{4}$$

De aquí sale *automáticamente* el mecanismo: si un punto está **estrictamente fuera del margen** y bien clasificado, el corchete es positivo, así que para cumplir la igualdad **obligatoriamente** $\alpha_i = 0$ y el punto no influye. Solo los puntos que *tocan o violan* el margen pueden tener $\alpha_i > 0$: esos son los vectores de soporte.

Tres tipos de punto. Las condiciones KKT clasifican cada muestra según su α_i:

Valor de α_i	**Posición del punto**	**Rol**
$\alpha_i = 0$	Fuera del margen, bien clasificado	No es vector de soporte (no influye)
$0 < \alpha_i < C$	Justo *sobre* el margen ($\xi_i = 0$)	Vector de soporte **libre** (fija b)
$\alpha_i = C$	Dentro del margen o mal clasificado ($\xi_i > 0$)	Vector de soporte **acotado**

Reconstrucción de la solución. El hiperplano se reconstruye *solo* a partir de los vectores de soporte. De la condición KKT $\mathbf{w} = \sum_i \alpha_i y_i \, \mathbf{x}_i$ y, como los $\alpha_i = 0$ no suman, queda

$$\mathbf{w} = \sum_{i \in \mathrm{SV}} \alpha_i y_i \, \mathbf{x}_i, \qquad b = y_k - \sum_{i \in \mathrm{SV}} \alpha_i y_i \, K(\mathbf{x}_i, \mathbf{x}_k), \tag{5}$$

donde b se calcula promediando sobre los vectores de soporte *libres* ($0 < \alpha_k < C$), que son los que caen exactamente sobre el margen. Esto justifica por qué "la solución solo depende de los vectores de soporte".

Predicción. Sustituyendo $\mathbf{w}$, la regla de decisión *nunca* usa $\mathbf{w}$ de forma explícita (necesario para el kernel): es una **suma ponderada solo sobre los vectores de soporte**:

$$f(\mathbf{x}) = \operatorname{sign}\Big(\sum_{i \in \mathrm{SV}} \alpha_i y_i \, K(\mathbf{x}_i, \mathbf{x}) + b \Big). \tag{6}$$

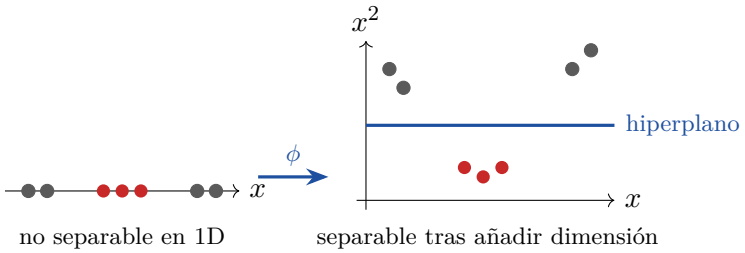
Cada vector de soporte "vota" con peso α_i según su parecido $K(\mathbf{x}_i, \mathbf{x})$ con la muestra nueva. Por eso menos vectores de soporte ⇒ predicción más rápida y, en general, mejor generalización.

En SVR el proceso es análogo: los puntos *dentro* del tubo ε cumplen la holgura complementaria con $\alpha_i = 0$ (no son vectores de soporte); solo los que tocan o salen del borde del tubo obtienen $\alpha_i > 0$ y construyen la función estimada.

4 El truco del kernel (*kernel trick*)

A veces **no existe** ningún hiperplano que separe las clases en el espacio original: decimos que las clases **no son linealmente separables**. El truco del kernel consiste en **inventar una dimensión nueva** (proyectar a un espacio de mayor dimensión $\phi(\mathbf{x})$) en la que las clases sí se puedan separar con un hiperplano. Lo potente es que **no hace falta calcular ϕ explícitamente**: basta con la función kernel

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i) \cdot \phi(\mathbf{x}_j).$$



no separable en 1D separable tras añadir dimensión

Kernel	$K(\mathbf{x}, \mathbf{x}')$	**Uso típico**
Lineal	$\mathbf{x} \cdot \mathbf{x}'$	Datos linealmente separables / muchas *features*
Polinómico	$(\gamma\, \mathbf{x} \cdot \mathbf{x}' + r)^d$	Fronteras curvas de grado d
RBF / Gauss	$\exp(-\gamma \lVert \mathbf{x} - \mathbf{x}' \rVert^2)$	**Opción por defecto**; fronteras muy flexibles
Sigmoide	$\tanh(\gamma\, \mathbf{x} \cdot \mathbf{x}' + r)$	Comportamiento similar a una red neuronal

El parámetro γ del RBF controla el *alcance* de cada punto: γ alto ⇒ fronteras muy locales (riesgo de *overfitting*); γ bajo ⇒ fronteras más suaves.

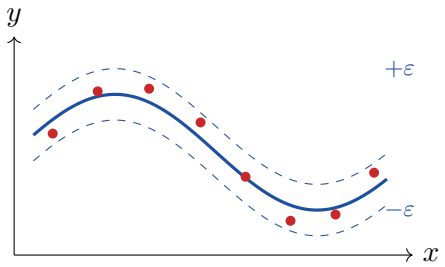
5 SVM para clasificación

Multiclase. La SVM es binaria por naturaleza. Para más de dos clases se combinan varios clasificadores: **one-vs-one** (uno por cada par de clases; el que usa `libsvm`) o **one-vs-rest** (uno por clase contra el resto).

Buena práctica. La SVM es **muy sensible a la escala** de las variables: siempre se deben estandarizar las *features* antes de entrenar. Los hiperparámetros se ajustan normalmente con búsqueda en malla + validación cruzada sobre C, γ y el kernel.

6 SVM para regresión (SVR)

El mismo concepto se adapta a la regresión: en vez de separar clases, la SVR ajusta una función dentro de un **"tubo" de ancho** ε alrededor de los datos. Los puntos que caen dentro del tubo **no se penalizan**; solo cuentan los que quedan fuera (que pasan a ser los vectores de soporte). Se pueden usar los mismos kernels (lineal, polinómico, RBF...) para estimar salidas no lineales.



SVR: se penaliza solo lo que sale del tubo de ancho ε.

7 Aplicaciones y rol histórico

Antes de la era del *deep learning*, las SVM se preferían frente a las redes neuronales porque sus matemáticas se entienden muy bien y la propiedad del **margen máximo** resultaba muy atractiva (las redes estándar de entonces cometían más errores de clasificación). Casos de éxito clásicos:

- Reconocimiento óptico de caracteres (OCR).
- Detección de rostros para el enfoque automático de cámaras digitales.
- Filtros de spam de correo electrónico.
- Clasificación de imágenes satelitales (nubes, tierra, agua, hielo...).

Actualmente las **redes neuronales profundas** suelen superar a las SVM en capacidad de aprendizaje y generalización, pero la SVM sigue siendo muy útil con **pocos datos** o **muchas dimensiones**.

8 Ventajas y desventajas

Ventajas	Desventajas
Muy eficaz en espacios de alta dimensión.	Costosa en datasets muy grandes ($\sim O(N^2)$–$O(N^3)$).
Robusta al *overfitting* con un C bien ajustado.	Sensible a la elección del kernel y de C, γ.
Solución determinista (óptimo global único).	Requiere estandarizar las variables.
Compacta: solo depende de los vectores de soporte.	Poco interpretable; no da probabilidades de forma directa.
Maneja no linealidad con el kernel trick.	Ajuste de hiperparámetros laborioso.

9 Código de referencia

Nota: esta sección es *solo de referencia*. Muestra cómo se invocaría la SVM en `scikit-learn`; no es necesario ejecutar nada en esta sesión.

Clasificación (referencial)

```python
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

# Escalado obligatorio + SVM con kernel RBF
clf = make_pipeline(
    StandardScaler(),
    SVC(kernel='rbf', C=1.0, gamma='scale') # C: regularizacion; gamma: alcance RBF
)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
```

Regresión SVR (referencial)

```python
from sklearn.svm import SVR

svr = SVR(kernel='rbf', C=100, gamma=0.1, epsilon=0.1) # epsilon = ancho del tubo
svr.fit(X_train, y_train)
```

Referencias

[1] C. Cortes, V. Vapnik. *Support-Vector Networks.* Machine Learning, 1995.

[2] B. Schölkopf, A. J. Smola. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond.* MIT Press, 2001.

[3] C. Bishop. *Pattern Recognition and Machine Learning.* Springer, 2006.

[4] S. Raschka, V. Mirjalili. *Python Machine Learning.* Packt Publishing, 2017.